Choose the Right LLM and Master RAG

Model selection, benchmarks, and embeddings
Topics: Model Selection, Benchmarks, Leaderboards, Vector Embeddings

Where We Are: **Course Progress**

Current day, completed topics, and upcoming modules

1. Day 11-12: Chat Completions API, Frontend Models, Tool Calling (completed)
2. Day 13: Hugging Face Transformers (completed)
3. Day 14: Model Selection & code generation (current)
4. Day 14-15: RAG (Retrieval Augmented Generation) (upcoming)
5. Ahead: Fine-tuning, Data Science, Final Project

Choose the **Right** LLM for Your Task

Stop asking for the single best model; match models to needs

Wrong question: "What's the best model?" Right question: "What's the right model for my task?"

Clarify the real decision to make when choosing an LLM.

Understand your requirements

List goals, constraints, and success criteria for your use case.

Review basic model information

Gather model specs, strengths, and limitations.

Analyze benchmarks

Compare performance on relevant tasks and metrics.

Test with your use case

Run real-world experiments to validate fit and quality.

What to Look For in an **AI Model**

Key attributes to evaluate before choosing a model

Open source vs. Closed source

Determine licensing and modification rights for your use case.

Chat vs. Reasoning vs. Hybrid models

Match model type to interaction and problem-solving needs.

Release date and knowledge cutoff

Know how current the model's training data is.

Number of parameters (bigger ≈ smarter)

Use parameter count as one signal of model capacity.

Training tokens (more data = more knowledge)

Evaluate training data scale to gauge knowledge coverage.

Context window size

Check how much input the model can consider at once.

Cost & Performance Factors

- **Cost: API pricing vs. compute costs**
  Compare per-call API fees against self-hosted compute expenses and scaling.
- **Speed: Response time measured in tokens per second**
  Measure throughput in tokens/sec for end-to-end responses.
- **Latency: Time to first token (TTFT)**
  Track time until first token is received from the model.
- **Rate limits: How many calls are allowed**
  Understand API or system call limits and throttling behavior.
- **Time to market: Build complexity impact**
  Assess integration effort and how it affects launch speed.
- **License: Usage restrictions and commercial limits**
  Review licensing terms for allowed use cases and commercial constraints.

Chinchilla **Scaling** Law: Optimal Model-Data Tradeoff

How parameters and training tokens must scale together

Training At Scale: Relationship Between Parameters And Training Data

Exact Link Between Model Size And Token Count For Optimal Training.

Rule Of Thumb: Doubling Parameters Requires Doubling Training Tokens.

Simple Guideline For Balancing Model Capacity And Data Volume.

Why It Matters Less Now: Better Compression Techniques, Inference-Time Improvements Such As Reasoning And RAG, And Smaller Models Getting Smarter

Emerging Methods Reduce Dependence On Sheer Scale.

Tough **Benchmarks** for Advanced Models

Key high-difficulty evaluations used to stress-test capabilities

1

GPQA (Google-Proof Q&A): 448 difficult physics, chemistry, biology questions; PhD level: 65% | Non-PhD: 34%; top models now ≈ 88%

A PhD-level set across STEM subjects with measured human baselines and leading-model performance.

2

MMLU Pro (Massive Multitask Language Understanding): 10 answer choices, tests broad knowledge and is harder than the original MMLU

Expanded-choice, broad-knowledge benchmark that raises difficulty over MMLU.

Hard **Benchmarks:** Advanced Problem Sets

High-level math and live coding challenges for skill validation

AIME (American Invitational Mathematics Examination): competitive high school math puzzles that test mathematical reasoning

Challenging math problems for top high school students.

Live Code Bench: coding problems from LeetCode and Codeforces, constantly updated to prevent contamination

Fresh coding tasks sourced and rotated regularly.

Tough AI Benchmarks: **Hard** Reasoning Tests

Deep-dive into extreme multi-step and superhuman evaluations

TRIAL

MUSA (Multi-step Soft Reasoning): tests reasoning through complex scenarios such as murder mystery whodunits, requiring means, motive, and opportunity

Evaluates multi-step causal and evidential reasoning in narrative scenarios.

HLE (Humanity's Last Exam): 2,500 superhuman-level questions designed to be extremely hard; models improved from 2–3% to 26%+ in months

Massive hard-question set tracking rapid model progress over time.

Benchmark Limitations That Matter to Practitioners

Why test scores can mislead decisions and how to read them critically

Training data contamination

models may have seen test questions during training

Inconsistent application

different hardware and self-reporting lead to incomparable results

Narrow scope

strong performance in one domain does not imply general PhD-level ability

Limited format

multiple choice formats miss nuance

Saturation

tests become too easy over time

Overfitting

selecting models optimized for specific benchmarks

Essential Leaderboards for AI Teams

Compare models, costs, and evaluation hubs at a glance

1

Artificial Analysis - artificialanalysis.ai

intelligence, speed, price comparisons, most comprehensive

2

Vellum - vellum.ai

API costs and context windows side-by-side

3

Scale AI (SEAL) - scale.com

specialized expert leaderboards

4

Hugging Face - huggingface.co/spac

open source model focus

5

Live Bench - livebench.ai

anti-contamination focus

Current Top **Models** (As of Recording)

Leading AI models to watch — verify live leaderboards

1. GPT-5 Codex (coding optimized)
2. GPT-5 High (max reasoning)
3. Grok-4
4. Claude 4.5 Sonnet
5. Gemini 2.5 Pro
6. GPT-Opus-1 R20B (first open source!)

Introduction to **RAG**

How retrieval augments LLM responses with external context

1

Problem: LLMs have knowledge cutoff dates, cannot access private or proprietary data, and are limited by context window size

Limitations that reduce relevance and accuracy over time and for private data

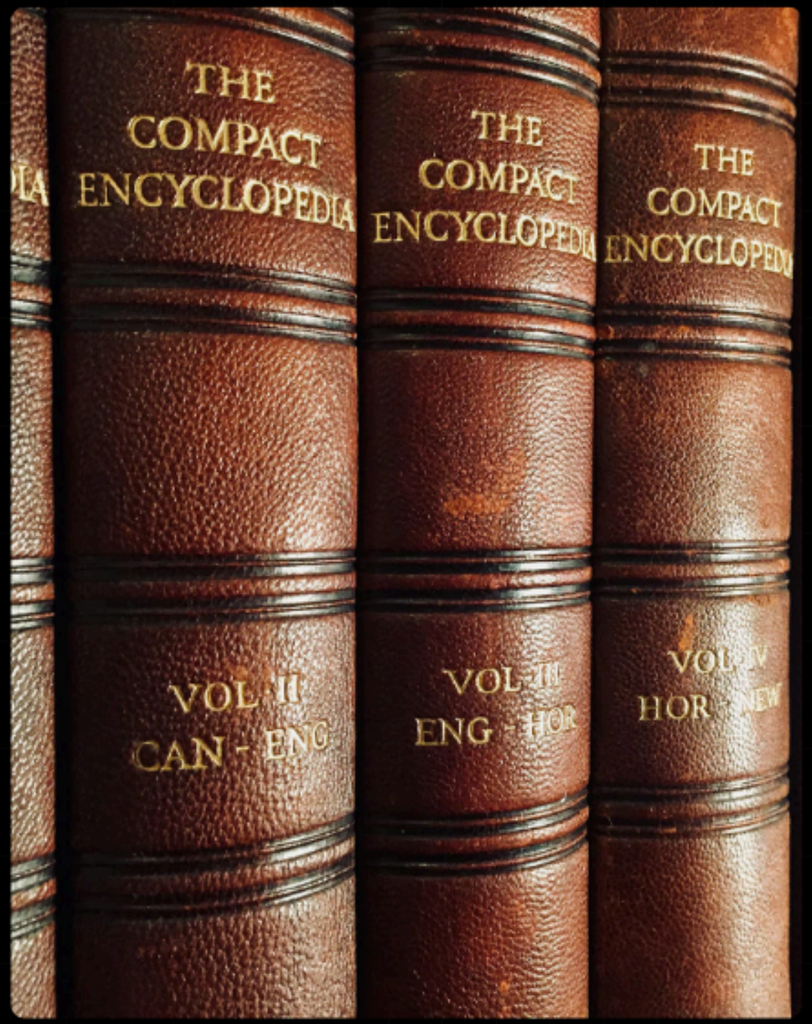
2

Solution: RAG (Retrieval Augmented Generation) augments prompts with relevant context retrieved from an external knowledge base so the model generates responses using both model knowledge and retrieved data

Combines model strengths with up-to-date, private, or large-context information

Simple **RAG** Example

Dictionary-based retrieval with exact-match limits



1

Example dictionary lookup

knowledge = {"lancaster": "Avery Lancaster, CEO of InsureElm", "car": "CarElm - auto insurance product"}

2

Limitations of exact string matching

Limitations: exact string matching only, brittle on typos and synonyms, fails on queries like "Who is Avery?" vs "Who is Lancaster?"

Understanding Two Types of LLMs

How autoregressive and encoder models serve different use cases

Autoregressive LLMs

- Autoregressive LLMs: predict the next token based on input; examples include GPT, Claude, Gemini; used for text generation, chat, and coding

Encoder LLMs

- Encoder LLMs: convert input into vectors; examples include BERT, OpenAI Embeddings, all-MiniLM; used for classification, embeddings, and semantic search

Understanding **Vector Embeddings**

How text becomes numerical meaning for search and comparison

1

Embeddings convert text into numbers that represent meaning so that similar meanings produce similar vectors close in vector space

Transforms semantic meaning into numeric vectors for comparison.

2

Example: "Ticket prices to London" -> 0.2, 0.8, 0.5, ... and "Flight costs to Heathrow" -> 0.21, 0.79, 0.51, ... showing semantic proximity

Shows how semantically similar phrases map to nearby vectors.

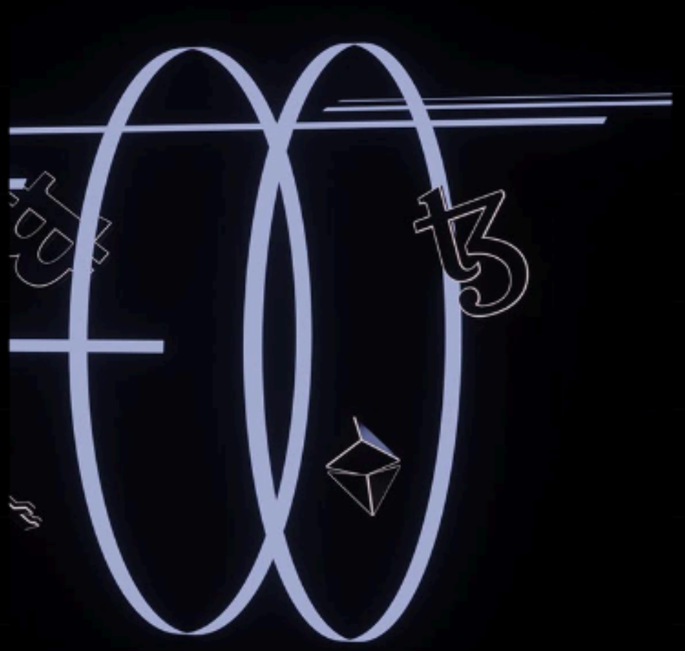
3

Dimensions typically range from hundreds to thousands per vector

High dimensionality captures nuanced meaning details.

Vector Math **Magic**

How embeddings capture and combine semantic relationships



Word2Vec analogies: King - Man + Woman = Queen

Demonstrates gender and royalty relationships in embedding space.

Geographic analogy: Paris - France + England = London

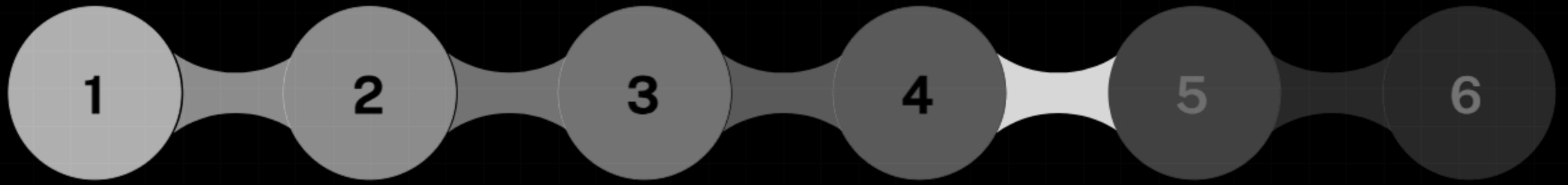
Shows how place and country relations appear in vectors.

Vectors capture semantic relationships and can be combined algebraically; modern encoders extend this to entire paragraphs

Embeddings scale from words to paragraphs for semantic tasks.

RAG with **Vectors**: How Retrieval-Augmented Generation Works

Turn user questions into accurate answers using semantic search



User Question

User asks a question, the question is encoded into a vector, a vector database is searched for similar vectors, matching text is retrieved, context is added to the prompt, and the LLM generates an accurate answer

Encode to Vector

User asks a question, the question is encoded into a vector, a vector database is searched for similar vectors, matching text is retrieved, context is added to the prompt, and the LLM generates an accurate answer

Vector Search

User asks a question, the question is encoded into a vector, a vector database is searched for similar vectors, matching text is retrieved, context is added to the prompt, and the LLM generates an accurate answer

Retrieve Matching Text

User asks a question, the question is encoded into a vector, a vector database is searched for similar vectors, matching text is retrieved, context is added to the prompt, and the LLM generates an accurate answer

Add Context to Prompt

User asks a question, the question is encoded into a vector, a vector database is searched for similar vectors, matching text is retrieved, context is added to the prompt, and the LLM generates an accurate answer

LLM Generates Answer

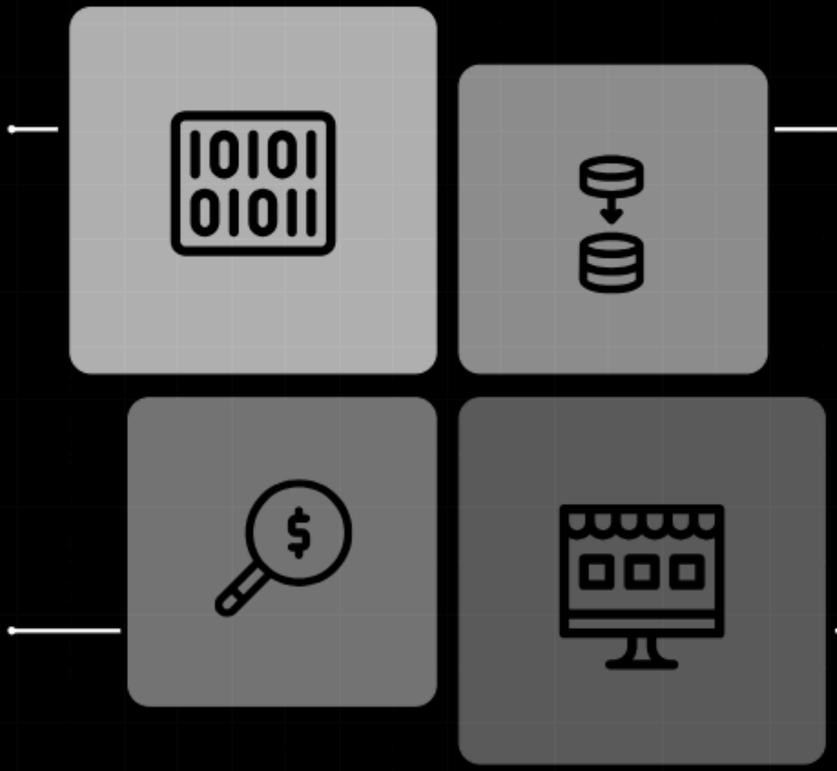
User asks a question, the question is encoded into a vector, a vector database is searched for similar vectors, matching text is retrieved, context is added to the prompt, and the LLM generates an accurate answer

Building a **Vector Data Store**

Key components and popular tools for vector search

Encoder Model

Components: encoder model to convert text into vectors, vector database to store vectors and original text, and similarity search (e.g., cosine similarity) to find closest vectors



Vector Database

Components: encoder model to convert text into vectors, vector database to store vectors and original text, and similarity search (e.g., cosine similarity) to find closest vectors

Similarity Search

Components: encoder model to convert text into vectors, vector database to store vectors and original text, and similarity search (e.g., cosine similarity) to find closest vectors

Popular Stores

Popular vector stores: Chroma, Pinecone, Weaviate, FAISS

Important **Clarification** on LLM Roles

How encoder and autoregressive models differ and interact

Encoder LLM: converts text to vectors and is used only for search, example model text-embedding-3-large

Autoregressive LLM: generates responses from natural language prompts and does not consume vectors directly, example models include GPT-4 and Claude

They do not talk to each other directly

RAG is a **Hack** (And That's OK!)

Practical, flexible retrieval that trades perfection for results

RAG is empirical and involves trial and error with many tricks to improve retrieval

It is not perfect but practical: better than nothing, cheaper than fine-tuning, flexible, updatable, and suitable for most use cases

Key **Takeaways** for RAG and Model Selection

Practical guidance to choose models and start RAG hands-on

No single "best" model exists; choose the right model for your task by checking basics, benchmarks, and leaderboards, and by testing with your use case

Evaluate fundamentals, public results, and real-world tests

RAG fundamentals: augment LLM knowledge with external data, vector embeddings enable semantic search, and RAG uses two LLMs (encoder for search, autoregressive for generation)

Combine retrieval plus generation for up-to-date, grounded responses

Next: hands-on RAG implementation with LangChain and Chroma

Practical implementation using popular open-source tools

Curated **Resources** for Further Reading

Key links to benchmarks, papers, and docs

Artificial Analysis: https://artificialanalysis.ai

Vellum LLM leaderboard: https://www.vellum.ai/llm-leaderboard

BENCH MARKING

Live Bench: https://livebench.ai

OpenAI Embeddings documentation: https://platform.openai.com/docs/gui

Chinchilla paper: https://arxiv.org/abs/2203.15556

Apple benchmark paper: https://arxiv.org/abs/2311.01964